



**BIT4133: NATURAL LANGUAGE PROCESSING WITH DEEP
LEARNING**

STUDENT NAME: IAN CHEGE

ADMISSION NUMBER: BSCCS/2024/32935

COURSE: BSc COMPUTER SCIENCE

UNIT CODE: BIT4133

**UNIT NAME: NATURAL LANGUAGE PROCESSING WITH DEEP
LEARNING**

LECTURER: MICHAEL NYORO

SEMESTER/ACADEMIC YEAR: SEMESTER 4.2/2025-2026

PROJECT AREA

AI BIBLE STUDY COMPANION: SCRIPTURE ANALYSIS USING NLP

**RESEARCH AREA: INFORMATION EXTRACTION & QUESTION
GENERATION**

TECHNOLOGIES USED

**PYTHON | NLTK | FASTAPI | REACT + VITE | OPENAI API (GPT-
4O-MINI) | RECHARTS**

GITHUB LINK: <https://github.com/Ian-Chege/NLP-Product>

TABLE OF CONTENT

WEEK 1: INTRODUCTION TO NLP AND APPLICATIONS	3
Introduction to NLP Concepts and Text Processing	3
Fig 1: Python Installation	3
Fig 2: Local Development Setup	3
Fig 3: NLTK Installation	3
Fig 4: Tokenization Output	4
Fig 5: Stopwords Removal Output	4
Fig 6: NLP Application Research Evidence	4
Student Reflection	5
WEEK 2: N-GRAM MODELS AND POS TAGGING	6
Language Modeling and Sentence Structure Analysis	6
Fig 1: Bigram or Trigram Output	6
Fig 2: POS Tagging Example	7
Fig 3: Python Code for POS Tagging	7
Fig 4: Language Prediction Example	8
Fig 5: Sentence Analysis Output	8
Student Reflection	9
WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING	10
Sequence Prediction and Hidden State Analysis	10
Fig 1: Sequence Labeling Code	10
Fig 2: Hidden Markov Model Output	11
Fig 3: Named Entity Recognition Example (NLTK)	11
Fig 4: Sentence Labeling Results	12
Fig 5: Dataset Used for Training or Testing	12
Bonus: Improved NER: spaCy vs NLTK	12
Student Reflection	13
WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS	14
Dependency Parsing and Semantic Similarity	14
Fig 1: Dependency Parsing Example	14
Fig 2: Dependency Parsing on Biblical Text	14
Fig 3: Assignment 1 Dependency Analysis	15
Fig 4: Semantic Similarity Analysis	15
Fig 5: Assignment 2 Similar vs Unrelated Sentence Pairs	15
Student Reflection	16
WEEK 5: MINI NLP PROJECTS	17
Complete NLP Pipeline and Mini Chatbot	17
Practical Task 1: Complete NLP Pipeline Output	17
Fig 2: Chatbot Demo Output	19
Fig 3: Interactive Chatbot Session	20
Mini Project: AI Bible Study Companion (jude-companion)	21
Student Reflection	23
WEEK 6: WORD EMBEDDINGS AND DISTRIBUTED REPRESENTATIONS	24
Practical task 1: Train a word2vec model	24
Practical task 2: Similarity analysis	24
Practical task 3: One-hot encoding vs word embeddings	25
New concept: Embedding-powered semantic search engine	26
Student Reflection	26

WEEK 1: INTRODUCTION TO NLP AND APPLICATIONS

Introduction to NLP Concepts and Text Processing

Fig 1: Python Installation

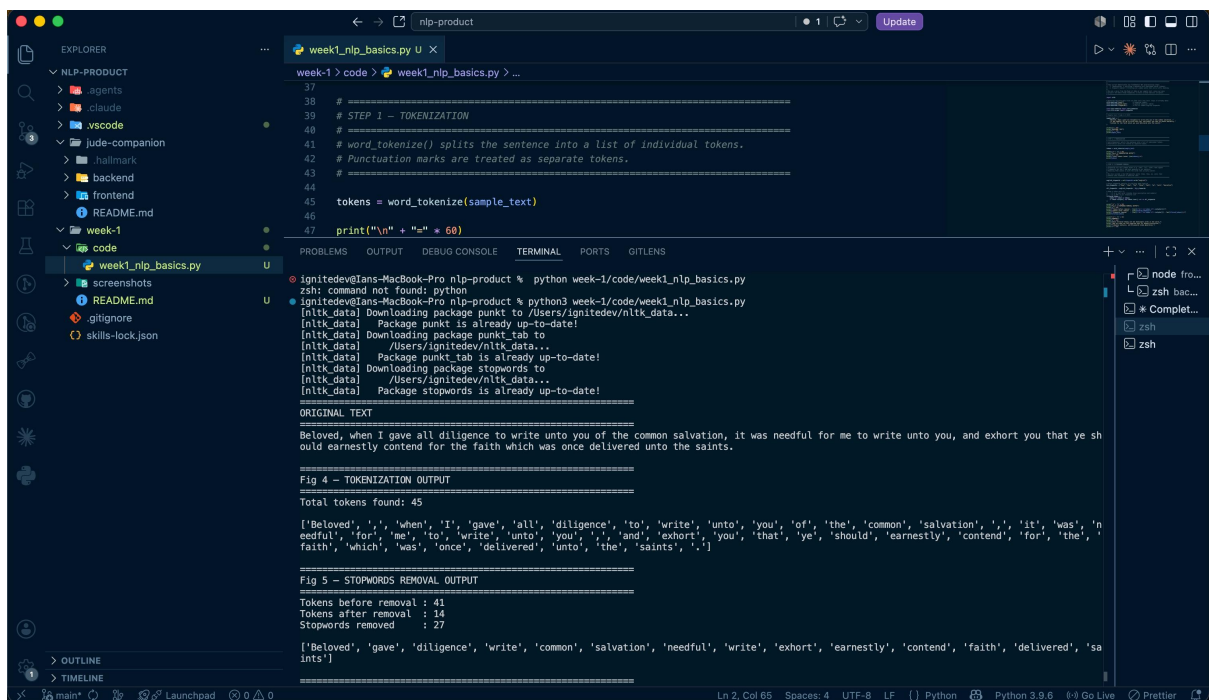


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

● ignitedev@Ians-MacBook-Pro nlp-product % python3 --version
Python 3.9.6
○ ignitedev@Ians-MacBook-Pro nlp-product %
```

Fig 1: Shows that Python is installed in the system.

Fig 2: Local Development Setup



```
week1_nlp_basics.py X
week-1 > code > week1_nlp_basics.py > ...

37 # =====
38 # STEP 1 - TOKENIZATION
39 # =====
40 # word_tokenize() splits the sentence into a list of individual tokens.
41 # Punctuation marks are treated as separate tokens.
42 # =====
43 tokens = word_tokenize(sample_text)
44
45 print("\n" + "-" * 60)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

● ignitedev@Ians-MacBook-Pro nlp-product % python week-1/code/week1_nlp_basics.py
zsh: command not found: python
● ignitedev@Ians-MacBook-Pro nlp-product % python3 week-1/code/week1_nlp_basics.py
[nltk_data] Downloading package punkt to /Users/ignitedev/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /Users/ignitedev/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /Users/ignitedev/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
=====
ORIGINAL TEXT
=====
Beloved, when I gave all diligence to write unto you of the common salvation, it was needful for me to write unto you, and exhort you that ye should earnestly contend for the faith which was once delivered unto the saints.

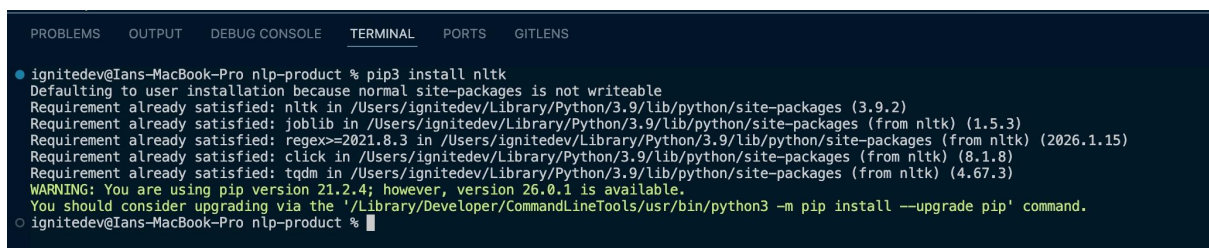
Fig 4 - TOKENIZATION OUTPUT
=====
Total tokens found: 45
['Beloved', ',', 'when', 'I', 'gave', 'all', 'diligence', 'to', 'write', 'unto', 'you', 'of', 'the', 'common', 'salvation', ',', 'it', 'was', 'needful', 'for', 'me', 'to', 'write', 'unto', 'you', 'and', 'exhort', 'you', 'that', 'ye', 'should', 'earnestly', 'contend', 'for', 'the', 'faith', 'which', 'was', 'once', 'delivered', 'unto', 'the', 'saints', '.']

Fig 5 - STOPWORDS REMOVAL OUTPUT
=====
Tokens before removal : 41
Tokens after removal : 14
Stopwords removed : 27

['Beloved', 'gave', 'diligence', 'write', 'common', 'salvation', 'needful', 'write', 'exhort', 'earnestly', 'contend', 'faith', 'delivered', 'sa', 'ints']
```

Fig 2: Shows local development setup in VS Code.

Fig 3: NLTK Installation



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

● ignitedev@Ians-MacBook-Pro nlp-product % pip3 install nltk
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: nltk in /Users/ignitedev/Library/Python/3.9/lib/python/site-packages (3.9.2)
Requirement already satisfied: joblib in /Users/ignitedev/Library/Python/3.9/lib/python/site-packages (from nltk) (1.5.3)
Requirement already satisfied: regex<=2021.8.3 in /Users/ignitedev/Library/Python/3.9/lib/python/site-packages (from nltk) (2026.1.15)
Requirement already satisfied: click in /Users/ignitedev/Library/Python/3.9/lib/python/site-packages (from nltk) (8.1.8)
Requirement already satisfied: tqdm in /Users/ignitedev/Library/Python/3.9/lib/python/site-packages (from nltk) (4.67.3)
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.
You should consider upgrading via the '/Library/Developer/CommandLineTools/usr/bin/python3 -m pip install --upgrade pip' command.
○ ignitedev@Ians-MacBook-Pro nlp-product %
```

Fig 3: Shows NLTK library installation.

Fig 4: Tokenization Output

```
=====
ORIGINAL TEXT
=====
Beloved, when I gave all diligence to write unto you of the common salvation, it was needful for me to write unto you, and exhort you that ye should earnestly contend for the faith which was once delivered unto the saints.

=====
Fig 4 - TOKENIZATION OUTPUT
=====
Total tokens found: 45

['Beloved', ',', 'when', 'I', 'gave', 'all', 'diligence', 'to', 'write', 'unto', 'you', 'of', 'the', 'common', 'salvation', ',', 'it', 'was', 'needful', 'for', 'me', 'to', 'write', 'unto', 'you', ',', 'and', 'exhort', 'you', 'that', 'ye', 'should', 'earnestly', 'contend', 'for', 'the', 'faith', 'which', 'was', 'once', 'delivered', 'unto', 'the', 'saints', '.']
```

Fig 4: *Demonstrates tokenization of a verse from Jude 1:3 using NLTK*

Fig 5: Stopwords Removal Output

```
=====
Fig 5 - STOPWORDS REMOVAL OUTPUT
=====
Tokens before removal : 41
Tokens after removal  : 14
Stopwords removed     : 27

['Beloved', 'gave', 'diligence', 'write', 'common', 'salvation', 'needful', 'write', 'exhort', 'earnestly', 'contend', 'faith', 'delivered', 'saints']
```

Fig 5: *Shows the token list after filtering out common English words (the, and, was) and KJV-specific words (unto, ye, thee).*

Fig 6: NLP Application Research Evidence

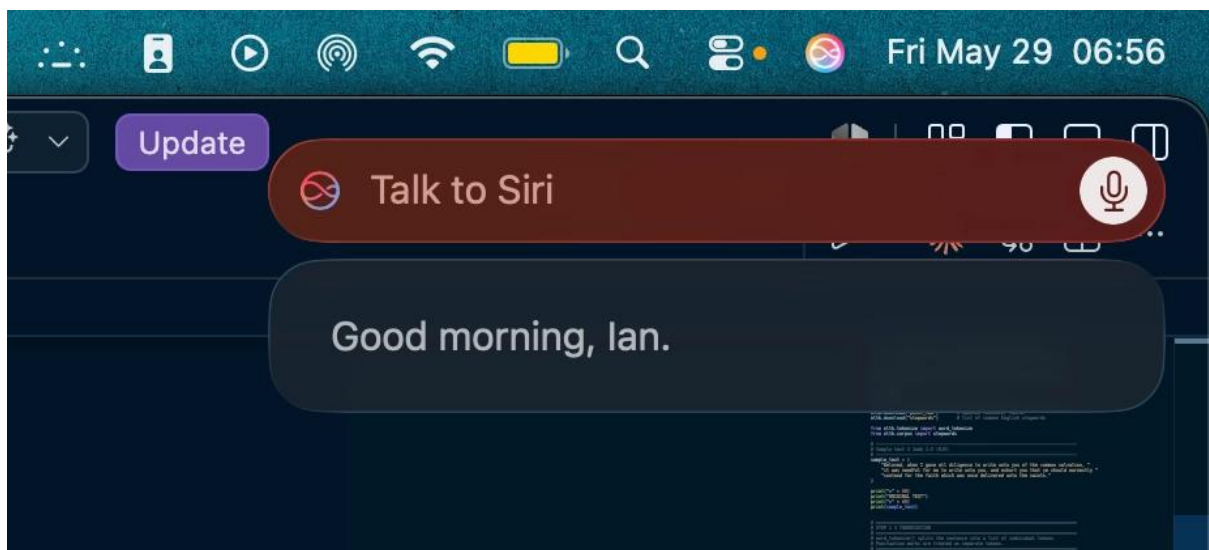


Fig 6: *Shows an interaction with Siri, an NLP application that uses speech recognition and intent detection to understand and respond to natural language in real time.*

Student Reflection

This week introduced core NLP concepts, tokenization and stopwords removal using Python and the NLTK library. I applied these techniques to a verse from the Book of Jude, the same text my main project analyses. Tokenization breaks a sentence into individual words so a program can process them one at a time. Stopword removal then filters out common words like 'the' and 'and', leaving only meaningful terms. I also explored real-world NLP applications such as Siri, which uses speech recognition and intent detection to understand and respond to natural language as shown in Fig 6 above.

WEEK 2: N-GRAM MODELS AND POS TAGGING

Language Modeling and Sentence Structure Analysis

Fig 1: Bigram or Trigram Output

```
ignitedev@Ians-MacBook-Pro nlp-product % python3 week-2/code/week2_ngrams_pos.py
=====
SAMPLE TEXT
=====
Beloved, when I gave all diligence to write unto you of the common salvation, it was needful for me to write unto you, and exhort you that ye should earnestly contend for the faith which was once delivered unto the saints. For there are certain men crept in unawares, who were before of old ordained to this condemnation, ungodly men, turning the grace of our God into lasciviousness, and denying the only Lord God, and our Lord Jesus Christ.
=====
Fig 1 - BIGRAM OUTPUT (top 10 most frequent)
=====
to      write      → 2x
write   unto       → 2x
unto    you       → 2x
Beloved when     → 1x
when    I       → 1x
I       gave    → 1x
gave    all     → 1x
all     diligence → 1x
diligence to     → 1x
you     of      → 1x
=====
Fig 1 - TRIGRAM OUTPUT (top 10 most frequent)
=====
to      write    unto    → 2x
write   unto     you     → 2x
Beloved when    I      → 1x
when    I       gave    → 1x
I       gave    all     → 1x
gave    all     diligence → 1x
all     diligence to     → 1x
diligence to     write  → 1x
unto    you     of      → 1x
you     of      the     → 1x
```

Fig 1: Shows the most frequent two-word (bigram) and three-word (trigram) sequences found in Jude 1:3-4.

Fig 2: POS Tagging Example

```
=====
Fig 2 – POS TAGGING EXAMPLE (first 20 words)
=====
```

Word	Tag	Label
Beloved	VRB	Verb
when	WRB	Other
I	PRP	Pronoun
gave	VBD	Verb
all	DT	Determiner
diligence	NN	Noun
to	TO	Other
write	VB	Verb
unto	IN	Preposition
you	PRP	Pronoun
of	IN	Preposition
the	DT	Determiner
common	JJ	Adjective
salvation	NN	Noun
it	PRP	Pronoun
was	VBD	Verb
needful	JJ	Adjective
for	IN	Preposition
me	PRP	Pronoun
to	TO	Other

Fig 2: Shows the first 20 words from the verse, each labelled with its grammatical role - Noun, Verb, Adjective, etc.

Fig 3: Python Code for POS Tagging

```
=====
Fig 3 – PYTHON CODE USED FOR POS TAGGING
=====

tagged = pos_tag(tokens)
# tokens is the list of words from word_tokenize()
# pos_tag returns a list of (word, tag) tuples
# e.g. [('Beloved', 'NNP'), ('gave', 'VBD'), ...]
```

Fig 3: Shows the core line of code used: `pos_tag(tokens)` which returns a list of (word, tag) tuples, e.g. `[('Beloved', 'NNP'), ('gave', 'VBD')]`.

Fig 4: Language Prediction Example

```
=====
Fig 4 – LANGUAGE PREDICTION EXAMPLE
=====
```

After 'unto', most likely next words:

- 'you' (2x in text)
- 'the' (1x in text)

After 'the', most likely next words:

- 'common' (1x in text)
- 'faith' (1x in text)
- 'saints' (1x in text)

After 'faith', most likely next words:

- 'which' (1x in text)

After 'God', most likely next words:

- 'into' (1x in text)
- 'and' (1x in text)

Fig 4: *Demonstrates simple language prediction using bigram frequencies. Given a word (e.g. 'unto'), the model looks at all bigrams starting with that word and returns the most likely word to follow.*

Fig 5: Sentence Analysis Output

```
=====
Fig 5 – SENTENCE ANALYSIS OUTPUT
=====
```

```
Total words (tokens) : 79
Unique words          : 57
Total bigrams         : 78
Total trigrams        : 77
```

Grammatical breakdown:

```
Verb          : 15
Preposition   : 13
Noun          : 10
Other         : 8
Pronoun       : 8
Determiner    : 7
Adjective     : 6
Proper Noun   : 6
Conjunction   : 3
Adverb       : 3
```

Fig 5: *Provides a complete structural breakdown of the text*

Student Reflection

This week introduced N-gram models and Part-of-Speech tagging using Python and NLTK. N-grams are sequences of consecutive words that reveal language patterns; bigrams capture two-word pairs while trigrams capture three-word sequences. I used these to build a simple next-word predictor on scripture from the Book of Jude. POS tagging then labelled each word with its grammatical role, showing which words are nouns, verbs, or adjectives. The main challenge was understanding the Penn Treebank tag codes, so I mapped them to plain English labels.

WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING

Sequence Prediction and Hidden State Analysis

Fig 1: Sequence Labeling Code

```
=====
Fig 1 - SEQUENCE LABELING CODE
=====

# Our own HMM - trains transition + emission probabilities from MACULA data
tagger = BiblicalHMM()
tagger.train(train_sentences)

# Each sentence is a list of (english_gloss, pos_tag) tuples
# e.g. [('jude', 'NNP'), ('servant', 'NN'), ('christ', 'NNP'), ...]

# Viterbi decoding finds the most likely tag sequence for new text
tokens = word_tokenize(sentence)
tagged = tagger.tag(tokens)

Training HMM on MACULA Greek NT... done.
```

Fig 1: Shows the code for our custom HMM tagger (*BiblicalHMM*). NLTK's built-in HMM tagger has a known numerical overflow bug on Python 3.9+.

Fig 2: Hidden Markov Model Output

```
=====
Fig 2 - HIDDEN MARKOV MODEL OUTPUT
=====
Input:
"Jude, the servant of Jesus Christ, and brother of James, to them that are sanctified by God the Father, and preserved in Jesus Christ, and called: Mercy unto you, and peace, and love, be multiplied."

Word          HMM Tag    Label
-----
Jude          CC         Conjunction
,             VB         Verb
the           DT         Determiner
servant       NN         Noun
of            IN         Preposition
Jesus         NNPN       Proper Noun
Christ        NNPN       Proper Noun
,             NNPN       Proper Noun
and           CC         Conjunction
brother       NN         Noun
of            IN         Preposition
James         NNPN       Proper Noun
,             VB         Verb
to            IN         Preposition
them          PRP        Pronoun
that          CC         Conjunction
are           VB         Verb
sanctified    VB         Verb
by            IN         Preposition
God           NN         Noun
the           DT         Determiner
Father        NN         Noun
,             PRP        Pronoun
and           CC         Conjunction
preserved     VB         Verb
in            IN         Preposition
Jesus         NNPN       Proper Noun
Christ        NNPN       Proper Noun
,             NNPN       Proper Noun
and           CC         Conjunction
called        VB         Verb
:             DT         Determiner
Mercy         NN         Noun
unto          IN         Preposition
you           PRP        Pronoun
,             VB         Verb
and           CC         Conjunction
peace         NN         Noun
,             PRP        Pronoun
and           CC         Conjunction
love          VB         Verb
,             PRP        Pronoun
be            VB         Verb
multiplied    VB         Verb
.             IN         Preposition
=====
```

Fig 2: Shows the HMM output on the first verse of Jude.

Fig 3: Named Entity Recognition Example (NLTK)

```
=====
Fig 3 - NAMED ENTITY RECOGNITION EXAMPLE
=====
Input: "Jude, the servant of Jesus Christ, and brother of James, to them that are sanctified by God the Father, and preserved in Jesus Christ, and called: Mercy unto you, and peace, and love, be multiplied."

[GPE] -> 'Jude'
[PERSON] -> 'Jesus Christ'
[PERSON] -> 'James'
[GPE] -> 'Father'
[GPE] -> 'Jesus Christ'
=====
```

Fig 3: Shows NER applied to the first verse using NLTK's `ne_chunk()`.

Fig 4: Sentence Labeling Results

```
=====
Fig 4 - SENTENCE LABELING RESULTS (full passage)
=====
Sentence 1: Jude [GPE], Jesus Christ [PERSON], James [PERSON], Father [GPE], Jesus Christ [GPE]
Sentence 2: Beloved [GPE]
Sentence 3: Lord God [ORGANIZATION], Lord Jesus Christ [ORGANIZATION]

Total entities found: 8
Jude [GPE]
Jesus Christ [PERSON]
James [PERSON]
Father [GPE]
Jesus Christ [GPE]
Beloved [GPE]
Lord God [ORGANIZATION]
Lord Jesus Christ [ORGANIZATION]
```

Fig 4: Shows NER across Jude 1:1-7. Lord Jesus Christ and Lord God are classified as ORGANIZATION rather than PERSON, a domain mismatch limitation, not a bug.

Fig 5: Dataset Used for Training or Testing

```
=====
Fig 5 - DATASET USED FOR TRAINING AND TESTING
=====
Training : MACULA Greek NT - expert-annotated New Testament
           (all NT books except Jude, trained word-for-word)
           Source: github.com/Clear-Bible/macula-greek
Testing  : Book of Jude, verses 1-7 (KJV)

Training sentences (NT minus Jude) : 7914
Jude verses (held-out test)       : 25

Sample training sentences:
=====
and/CC if/CC anyone/PRP away/VB from/IN the/DT words/NN the/DT book/NN ...
says/VB the/DT testifying/VB things/PRP yes/RP coming/VB quickly/JJ amen/RP come/VB ...
the/DT grace/NN the/DT lord/NN jesus/NNP with/IN all/JJ ...
```

Fig 5: Shows the datasets used, MACULA Greek NT is expert-annotated NT text with verified POS tags, trained on all NT books (7,914 verses) with Jude held out as the test set.

Bonus: Improved NER: spaCy vs NLTK

NLTK's ne_chunk was trained on news text, causing Jude and Jesus Christ to be labelled as GPE (place), and Lord Jesus Christ as ORGANIZATION. To demonstrate a better approach, the same passage was run through spaCy's en_core_web_sm: a neural NER model trained on the OntoNotes 5 corpus (~1 million words, diverse genres).

Key Observations: spaCy correctly identifies Jude, Jesus Christ, and James as PERSON.

Both models miss titles like Lord God and Lord Jesus Christ too domain-specific for any modern-trained NER model.

Student Reflection

This week introduced Hidden Markov Models and sequence labeling. Training on MACULA Greek NT expert-annotated biblical text produced correctly varied POS tags. For NER, NLTK's `ne_chunk` misclassified Jude as a place and Lord Jesus Christ as an organisation because it was trained on news text. Replacing it with spaCy corrected Jude and Jesus Christ to PERSON. Both models missed titles like Lord God, showing that even better models have limits when applied to domain-specific ancient text.

WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS

Dependency Parsing and Semantic Similarity

Fig 1: Dependency Parsing Example

```
=====
Fig 1 – DEPENDENCY PARSING EXAMPLE
=====
Input: "The lecturer teaches Natural Language Processing."

Word          Dep Role      Head Word
-----
The           det          lecturer
lecturer     nsubj        teaches
teaches       ROOT         teaches
Natural       compound    Language
Language      compound    Processing
Processing    dobj         teaches
.             punct       teaches

Key relationships:
nsubj  → 'lecturer' (head: 'teaches')
ROOT   → 'teaches' (head: 'teaches')
dobj   → 'Processing' (head: 'teaches')
```

Fig 1: shows the output of spaCy's dependency parser

Fig 2: Dependency Parsing on Biblical Text

```
=====
Fig 2 – DEPENDENCY PARSING ON BIBLICAL TEXT (Jude 1:3)
=====
Input: "Jude, the servant of Jesus Christ, contend for the faith."

Word          Dep Role      Head Word
-----
Jude           nsubj        contend
,             punct        Jude
the            det          servant
servant        appos        Jude
of             prep         servant
Jesus          compound    Christ
Christ         pobj        of
,             punct        Jude
contend        ROOT         contend
for            prep         contend
the            det          faith
faith         pobj        for
.             punct        contend
```

Fig 2: applies the same parser to Jude 1:3

Fig 3: Assignment 1 Dependency Analysis

```
=====
Fig 3 - ASSIGNMENT 1: DEPENDENCY ANALYSIS
=====

Sentence: "The administrator updated student records."
Word      Dep Role      Head Word
-----
The        det          administrator
administrator nsubj       updated
updated    ROOT         updated
student    compound    records
records    dobj         updated
.          punct        updated

Summary → Subject: 'administrator' | Verb: 'updated' | Object: 'records'

Sentence: "Machine learning improves language processing."
Word      Dep Role      Head Word
-----
Machine    compound    learning
learning   nsubj       improves
improves   ROOT         improves
language   compound    processing
processing dobj         improves
.          punct        improves

Summary → Subject: 'learning' | Verb: 'improves' | Object: 'processing'
```

Fig 3: Shows dependency analysis for two sentences

Fig 4: Semantic Similarity Analysis

```
=====
Fig 4 - SEMANTIC SIMILARITY ANALYSIS
=====

/Users/ignitedev/personal/nlp-product/week-4/code/week4_syntax_semantics.py:140: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.
  score = doc1.similarity(doc2)

Sentence A: "The student passed the examination."
Sentence B: "The learner succeeded in the exam."
Similarity Score: 0.8251 (high - similar meaning)
```

Fig 4: Shows spaCy comparing 2 sentences

Fig 5: Assignment 2 Similar vs Unrelated Sentence Pairs

```
=====
Fig 5 - ASSIGNMENT 2: SIMILAR VS UNRELATED SENTENCE PAIRS
=====

/Users/ignitedev/personal/nlp-product/week-4/code/week4_syntax_semantics.py:168: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.
  score = doc1.similarity(doc2)

[Similar pair]
Sentence A: "Jude urges believers to contend for the faith."
Sentence B: "Jude encourages Christians to defend their belief."
Similarity Score: 0.7636

[Unrelated pair]
Sentence A: "The market price of wheat fell sharply."
Sentence B: "The disciples gathered in the upper room."
Similarity Score: 0.5282

Observation:
The similar pair scores higher because both sentences share related vocabulary (faith/belief, urge/encourage, Jude/Jude).
The unrelated pair scores lower because wheat markets and biblical gatherings share no common vocabulary or context.
```

Fig 5: Shows the contrast between two pairs of sentences

Student Reflection

This week introduced dependency parsing and semantic similarity using spaCy. Dependency parsing reveals sentence structure by labelling each word with its grammatical role and linking it to the word it depends on, making it possible to extract subject, verb, and object automatically. Semantic similarity goes further by comparing meaning rather than exact words; two sentences can score highly even when phrased completely differently. Applying these tools to the Book of Jude highlighted a challenge: archaic KJV phrasing causes some dependency labels to differ from what a modern English parser expects.

WEEK 5: MINI NLP PROJECTS

Complete NLP Pipeline and Mini Chatbot

Practical Task 1: Complete NLP Pipeline Output

```
○ ignitedev@Ians-MacBook-Pro nlp-product % python3 week-5/code/week5_pipeline_chatbot.py
/Users/ignitedev/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only
supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3
/urllib3/issues/3020
  warnings.warn(
=====
PRACTICAL TASK 1 - COMPLETE NLP PIPELINE
=====

Input text:
"Natural Language Processing helps computers understand human language by breaking sentences into words and analysing
their grammatical roles."

Step 1 - TOKENIZATION
=====
Total tokens : 19
Token list : ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'by',
'breaking', 'sentences', 'into', 'words', 'and', 'analysing', 'their', 'grammatical', 'roles', '.']

Step 2 - STOPWORD REMOVAL
=====
Before removal : 18 words
After removal : 14 words
Filtered list : ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'brea
king', 'sentences', 'words', 'analysing', 'grammatical', 'roles']
```

Step 3 - POS TAGGING

Word	Tag	Label
Natural	JJ	Adjective
Language	NNP	Proper Noun
Processing	NNP	Proper Noun
helps	VBZ	Verb
computers	NNS	Noun
understand	VBP	Verb
human	JJ	Adjective
language	NN	Noun
breaking	VBG	Verb
sentences	NNS	Noun
words	NNS	Noun
analysing	VBG	Verb
grammatical	JJ	Adjective
roles	NNS	Noun

Grammatical breakdown:

Noun	: 5
Verb	: 4
Adjective	: 3
Proper Noun	: 2

Step 4 - N-GRAMS

```
Total bigrams : 13
Total trigrams : 12
Sample bigrams : [('Natural', 'Language'), ('Language', 'Processing'), ('Processing', 'helps'), ('helps', 'computers'
), ('computers', 'understand')]
Sample trigrams : [('Natural', 'Language', 'Processing'), ('Language', 'Processing', 'helps'), ('Processing', 'helps',
'computers')]
```

Step 5 – DEPENDENCY PARSING

Word	Dep Role	Head Word
Natural	compound	Language
Language	compound	Processing
Processing	nsubj	helps
helps	ROOT	helps
computers	nsubj	understand
understand	ccomp	helps
human	amod	language
language	dobj	understand
by	prep	understand
breaking	pcomp	by
sentences	dobj	breaking
into	prep	breaking
words	pobj	into
and	cc	breaking
analysing	conj	breaking
their	poss	roles
grammatical	amod	roles
roles	dobj	analysing
.	punct	helps

Key roles → Subject: 'Processing' | Verb: 'helps' | Object: 'language'
/Users/ignitedev/personal/nlp-product/week-5/code/week5_pipeline_chatbot.py:140: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.
score = sentence_a.similarity(sentence_b)

Step 6 – SEMANTIC SIMILARITY

Sentence A : "NLP helps computers understand language."
Sentence B : "Artificial intelligence enables machines to process text."
Score : 0.6537 (moderate similarity)

The screenshots above shows the complete NLP pipeline outputs from the terminal

Fig 2: Chatbot Demo Output

```
=====
PRACTICAL TASK 2 – MINI CHATBOT SIMULATION
=====
Theme: Self-Driving Cars | AutoBot – your autonomous vehicle guide
=====
Demo: running through a simulated conversation

You    : Hello!
AutoBot: Hello, I'm AutoBot, your self-driving car guide. Ask me anything about autonomous vehicles – how they work,
safety, or when they'll be on every road.

You    : How exactly do self-driving cars detect the road?
AutoBot: Self-driving cars use a combination of cameras, radar, LiDAR, and ultrasonic sensors to build a 360-degree p
icture of the road. An onboard AI processes this data in real time to steer, brake, and make decisions – all without a
human touching the wheel.

You    : Are they actually safe though?
AutoBot: Safety is the core promise. Autonomous vehicles don't get tired, distracted, or drunk. Studies show that ove
r 90% of road accidents are caused by human error – self-driving technology directly targets that statistic. That said,
the technology is still maturing and edge cases like bad weather remain active research challenges.

You    : Which companies are building them?
AutoBot: The biggest names right now are Waymo (Google's spin-off), Tesla, Cruise (GM), and Mobileye. Waymo currently
runs fully driverless robotaxi services in San Francisco and Phoenix. Tesla calls its system Full Self-Driving but sti
ll requires driver supervision.

You    : What are the levels of autonomy?
AutoBot: There are six levels of autonomy defined by SAE International: Level 0 is fully manual. Level 2 (like Tesla
Autopilot) assists but the driver must stay alert. Level 4 can handle most conditions without a driver. Level 5 is full
autonomy in all conditions – nobody has reached that yet.

You    : Is it even legal to use them?
AutoBot: Regulations vary by country. In the US, California and Arizona allow commercial robotaxi operations. The EU
is developing a unified framework. Most countries require a safety driver or restrict testing to designated zones. Full
public deployment is still years away in most places.

You    : When will everyone have one?
AutoBot: Level 4 vehicles are already operating in limited areas today. Widespread consumer availability depends on r
egulation, cost, and edge-case safety. Most industry experts expect broad deployment in cities by the early 2030s, with
rural and highway coverage following after.

You    : Wow! Thanks!
AutoBot: Thanks for chatting! The road to fully autonomous vehicles is being paved one algorithm at a time. Stay curi
ous – and buckle up!
```

Fig 2 shows the chatbot simulation

Fig 3: Interactive Chatbot Session

```
=====
MINI PROJECT – STUDENT ACADEMIC ASSISTANT CHATBOT
=====
Type a message and press Enter. Type 'bye' to exit.

You: hi
Bot: Hello! I'm your Student Academic Assistant. I can help with exam prep, course content, assignments, and study tips for the NLP unit. What would you like to know?

You: 

```



```
=====
MINI PROJECT – STUDENT ACADEMIC ASSISTANT CHATBOT
=====
Type a message and press Enter. Type 'bye' to exit.

You: hi
Bot: Hello! I'm your Student Academic Assistant. I can help with exam prep, course content, assignments, and study tips for the NLP unit. What would you like to know?

You: best practices for this unit
Bot: Unit registration is handled through the student portal. Deadlines are usually in the last week of the first month of a semester. If you're having trouble, visit the registrar's office directly.

You: when are the cats?
Bot: I'm not sure about that one. Try asking me about the CAT exam, unit registration, the mini project, or NLP study tips.

You: I said tell me about exams
Bot: I'm not sure about that one. Try asking me about the CAT exam, unit registration, the mini project, or NLP study tips.

You: okay CAT exam
Bot: CAT 1 covers everything from Weeks 1 to 5: tokenization, stopword removal, POS tagging, N-grams, HMMs, dependency parsing, and semantic similarity. Make sure you can explain each concept and read the output your scripts produce.

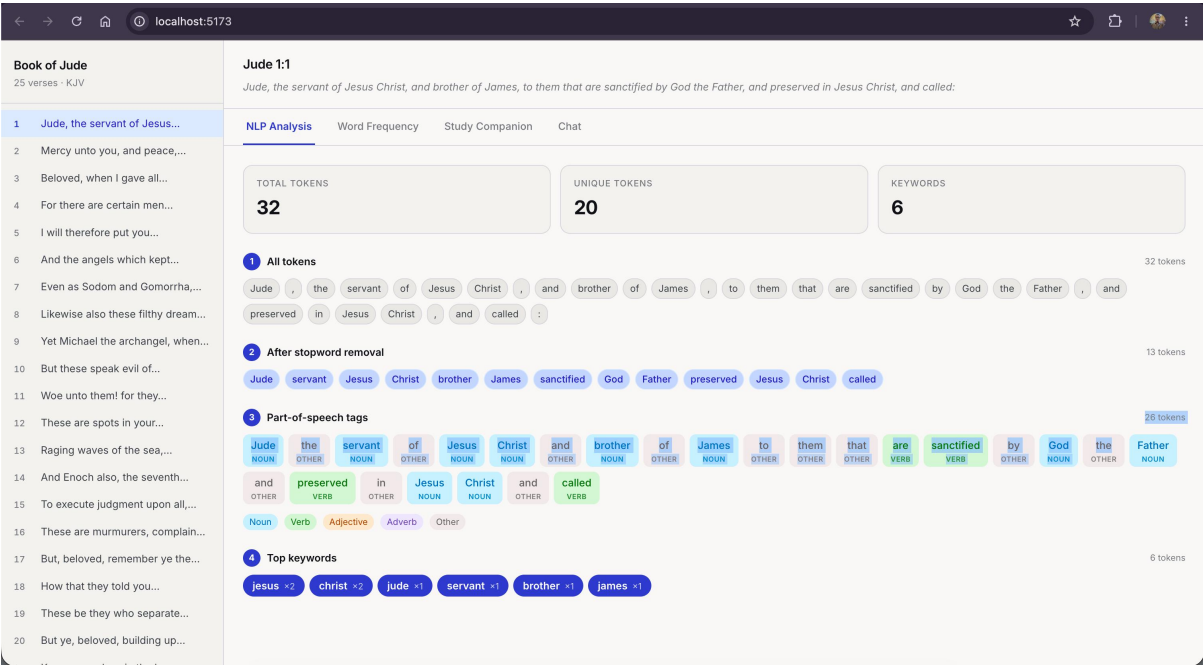
You: okay, thank you
Bot: Good luck with your studies! You've got this. Remember – understanding the output matters more than memorising the code.

You: bye
Bot: Good luck with your studies! You've got this. Remember – understanding the output matters more than memorising the code.
```

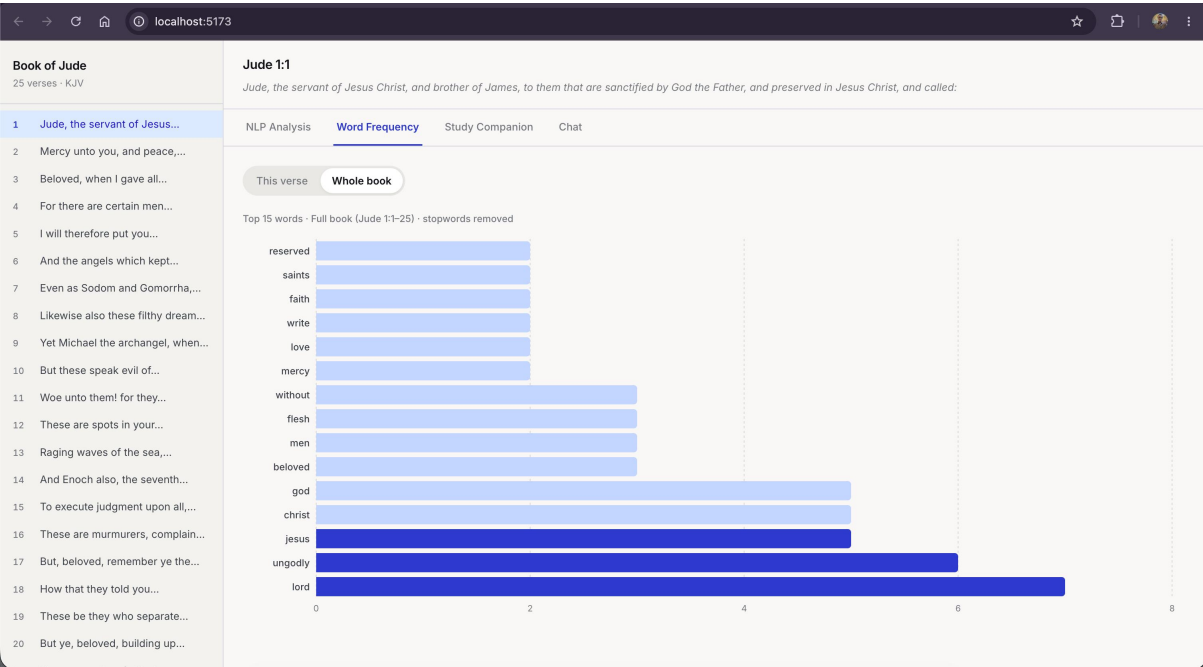
Fig 3 shows the chatbot running in interactive mode where the user types their own messages to the chatbot

Mini Project: AI Bible Study Companion (jude-companion)

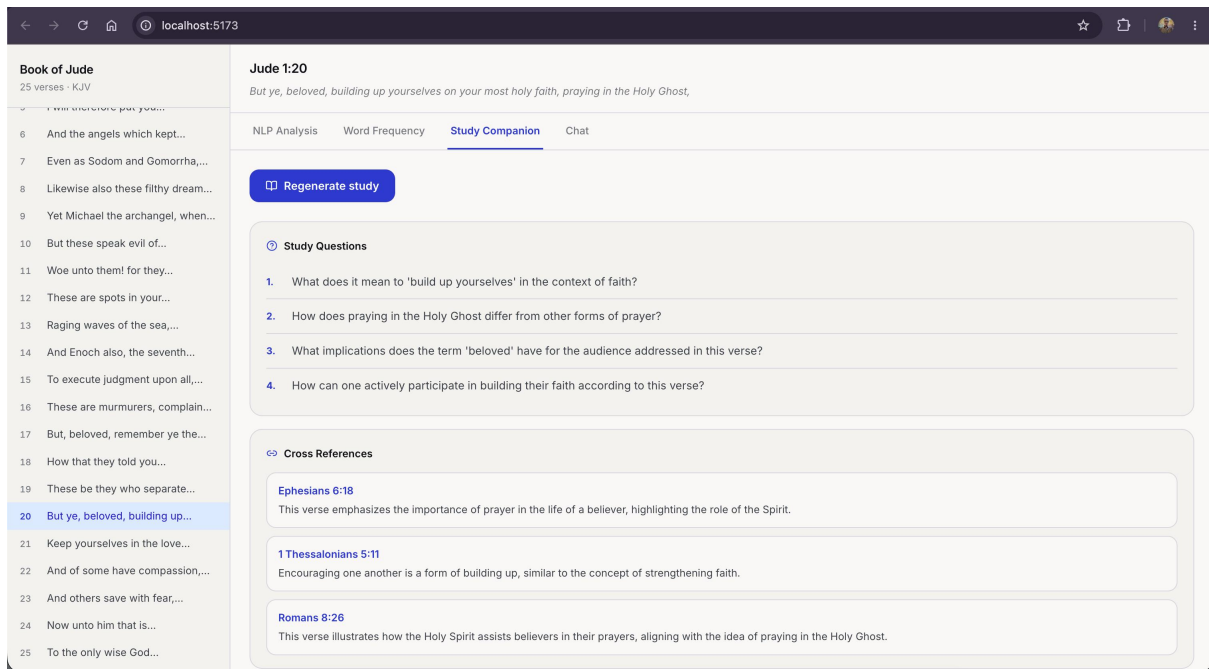
AI Bible Study Companion is a web application that applies NLP techniques to the Book of Jude (KJV)



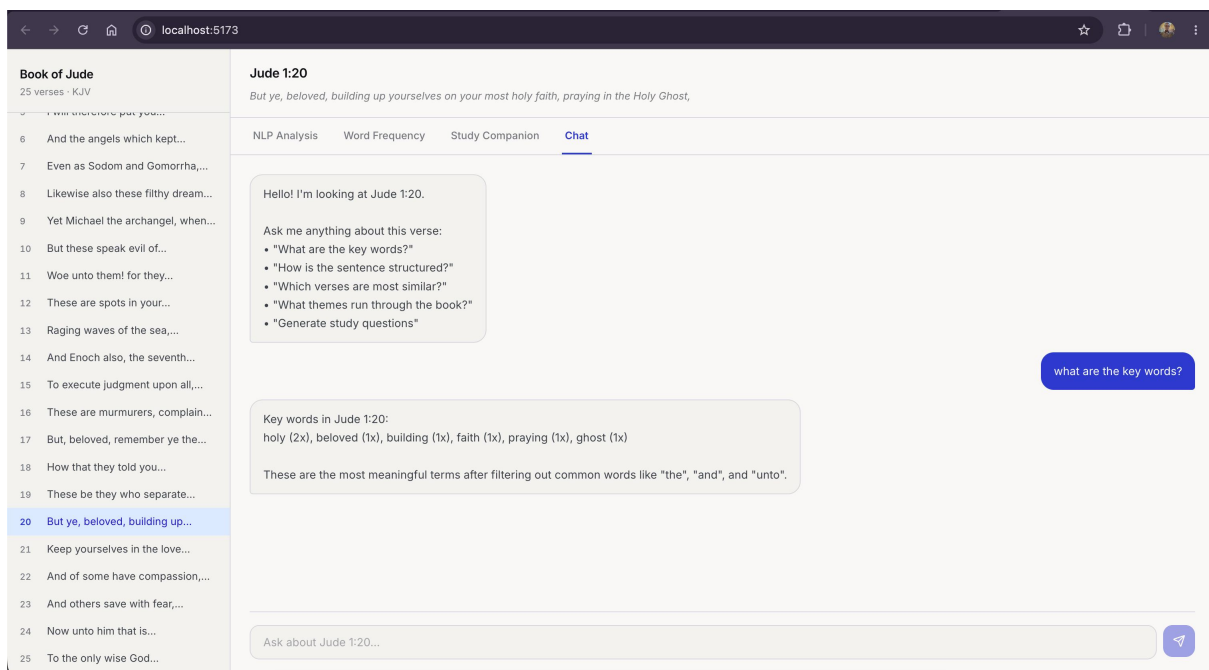
The NLP Analysis tab shows the full processing pipeline applied to the selected verse



The Word Frequency tab renders a horizontal bar chart of the most repeated meaningful words



The Study Companion tab uses GPT-4o-mini to generate AI-powered study material for the selected verse.



The Chat tab provides a conversational interface to the NLP backend

Student Reflection

Week 5 brought together every technique from the course into a single working pipeline. Running tokenization, stopword removal, POS tagging, n-grams, dependency parsing, and semantic similarity. The mini chatbot showed how rule-based NLP systems work at a basic level, matching user input to intents using keyword detection.

WEEK 6: WORD EMBEDDINGS AND DISTRIBUTED REPRESENTATIONS

Practical task 1: Train a word2vec model

```
=====
PRACTICAL TASK 1 – TRAIN A WORD2VEC MODEL
=====

Vocabulary size : 11 words
Vector size      : 50 dimensions

Vector for 'nlp' (first 8 of 50 values):
[-0.016  0.009 -0.008  0.002  0.017 -0.009  0.009 -0.014] ...

Words most similar to 'nlp':
love      → 0.1249
modern    → 0.0740
learning  → 0.0424
```

Training a word2vec model

Practical task 2: Similarity analysis

```
=====
PRACTICAL TASK 2 – SIMILARITY ANALYSIS (10 WORDS)
=====
```

Word	Nearest neighbour	Cosine score
man	woman	0.8141
woman	girl	0.9057
school	college	0.8148
money	excuse	0.7976
city	yankees	0.8273
water	hot	0.8158
music	jazz	0.8199
war	civil	0.7975
science	historical	0.8968
government	federal	0.8838

```
Direct similarity('man', 'woman') = 0.8141
```

```
=====
WORD ANALOGY – king – man + woman ≈ ?
=====
```

```
king – man + woman is closest to:
```

```
richard  → 0.8064
smith    → 0.8056
queen    → 0.8038
```

Word2Vec (Skip-Gram, 100-dim) trained on the Brown corpus, The model learned real semantic relationships

Practical task 3: One-hot encoding vs word embeddings

=====

PRACTICAL TASK 3 – ONE-HOT ENCODING vs WORD EMBEDDINGS

=====

One-Hot Encoding (each word is an isolated 1):
cat → [1, 0, 0]
dog → [0, 1, 0]
animal → [0, 0, 1]

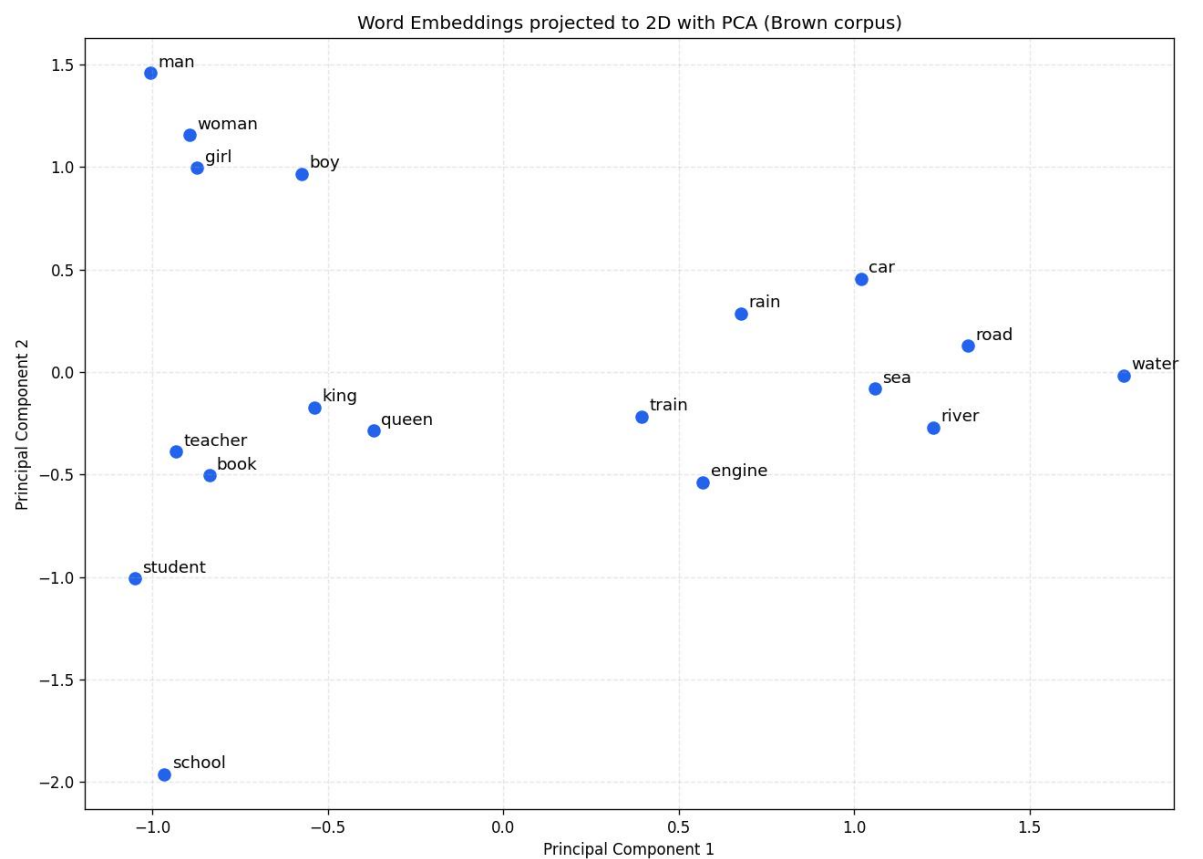
Word Embedding (dense, learned from context – first 6 dims):
cat → [0.021 0.225 -0.103 0.2 -0.05 -0.26] ...
dog → [-0.065 0.069 -0.085 0.218 -0.044 -0.108] ...
animal → [-0.047 0.067 -0.018 0.046 -0.143 -0.271] ...

COMPARISON REPORT

Feature	One-Hot Encoding	Word Embeddings
Size	Vocab length	Fixed & compact
Meaning representation	None	Captures meaning
Efficiency	Sparse / wasteful	Dense / efficient
Semantic understanding	No	Yes

Conclusion: One-Hot vectors grow with the vocabulary and treat every word as unrelated. Embeddings are compact, fixed-size, and place words with similar meaning close together in vector space.

One-Hot Encoding versus Word Embeddings: sparse isolated vectors with no meaning, contrasted with dense vectors that capture semantic relationships



100-dim Brown-corpus embeddings projected to 2D with PCA to depict similarity

New concept: Embedding-powered semantic search engine

Book of Jude

25 verses - KJV

1

Jude, the servant of Jesus...

2

Mercy unto you, and peace,...

3

Beloved, when I gave all...

4

For there are certain men...

5

I will therefore put you...

6

And the angels which kept...

7

Even as Sodom and Gomorraha,...

8

Likewise also these filthy dream...

9

Yet Michael the archangel, when...

10

But these speak evil of...

11

Woe unto them! for they...

12

These are spots in your...

13

Raging waves of the sea,...

14

And Enoch also, the seventh...

15

To execute judgment upon all,...

16

These are murmurers, complain...

17

But, beloved, remember ye the...

18

How that they told you...

19

These be they who separate...

20

But ye, beloved, building up...

Jude 1:1

Jude, the servant of Jesus Christ, and brother of James, to them that are sanctified by God the Father, and preserved in Jesus Christ, and called:

NLP Analysis

Semantic Search

Word Frequency

Study Companion

Chat

Q

Semantic Search — find verses by meaning

false teachers sneaking in

false teachers sneaking in

God keep me from falling

angels who left their home

glory and majesty to God forever

Ranked by meaning · cosine similarity of word-embedding vectors

Jude 1:4

0.630

For there are certain men crept in unawares, who were before of old ordained to this condemnation, ungodly men, turning the grace of our God into lasciviousness, and denying the only Lord God, and our Lord Jesus Christ.

Jude 1:8

0.541

Likewise also these filthy dreamers defile the flesh, despise dominion, and speak evil of dignities.

Jude 1:6

0.530

And the angels which kept not their first estate, but left their own habitation, he hath reserved in everlasting chains under darkness unto the judgment of the great day.

Jude 1:9

0.512

Yet Michael the archangel, when contending with the devil he disputed about the body of Moses, durst not bring against him a railing accusation, but

Concept applied to the AI Bible Study Companion: an embedding-powered semantic search. Each verse is encoded as a document embedding (the average of its word vectors); the query is embedded the same way and verses are ranked by cosine similarity. The query 'false teachers sneaking in' returns Jude 1:4 despite sharing no words with it matching by meaning rather than keywords.

Student Reflection

This week introduced word embeddings. Earlier methods like One-Hot Encoding treat every word as an isolated symbol, producing large sparse vectors with no sense of meaning. Word embeddings instead represent each word as a compact dense vector learned from context, so words used in similar ways end up close together. As the creative task, I applied embedding-powered semantic search engine to my main project by adding a semantic search feature to the AI Bible Study Companion: a query like "false teachers sneaking in" correctly returns Jude 1:4 about men who "crept in unawares", even though they share no words, because the match is made by meaning rather than keywords.